

A Multilingual Evaluation of Large Language Models for Data Analytics Code Generation

DS 5500: Data Science Capstone
Methodology Report

Reghunaath Ajith Kumar Ahila & Naga Pavithra Lagisetty

DS 5500: Data Science Capstone
Khoury College of Computer Sciences
Northeastern University, Boston, MA
March 21, 2026

Contents

1	Introduction	2
2	Purpose of the Methodology	2
3	Problem Statement	3
3.1	Defining the Problem	3
3.2	The Research Gap: Data Analysis Code Generation	3
3.3	Significance	4
4	Data Collection and Preparation	5
4.1	Data Sources and Design Rationale	5
4.2	Database Description	5
4.3	Query Design: 50 Query Benchmark Suite	6
4.4	Language Specific Execution Environments	6
5	Selection of LLM Models	7
5.1	Models Considered	7
5.2	Final Model Selection	7
6	Model Development and Evaluation Framework	8
6.1	API Configuration	8
6.2	Evaluation Pipeline	9
6.3	Prompt Engineering Decisions	9
7	Results and Evaluation	10
7.1	Pilot Results — Queries 1–10 (Tiers 1–2)	10
7.2	Performance Visualisation — Confirmed Pilot Results	10
7.3	Full Evaluation Plan	11
7.4	Pilot Findings	11
8	Conclusion	12
9	DataLens: Applied Web Application	13
9.1	Motivation	13
9.2	Core Features	13
9.3	System Architecture	14
9.4	Connection to Benchmark Findings	14

1 Introduction

Data analysts in academia and industry routinely work across three programming environments: **SQL** for relational database queries, **Python (Pandas)** for tabular data manipulation, and **R (dplyr/tidyverse)** for statistical analysis and reporting. The rapid adoption of large language model (LLM) assistants has created a new expectation that an analyst can describe a question in plain English and receive correct, executable analysis code in whichever language their pipeline requires.

This study benchmarks four frontier LLMs - **Google Gemini 3.1 Pro Preview**, **OpenAI GPT-5.4**, **Anthropic Claude Opus 4**, and **Perplexity Sonar Reasoning** - across all three languages, yielding **twelve model language evaluation tracks** (4 models $\times$ 3 languages).

A central design decision was to **construct an entirely novel benchmark** rather than rely on any pre existing dataset. We built a custom **17-table university relational database** with hand authored schema and synthetic data, then composed **50 natural language queries** across five complexity tiers. This approach ensures that evaluation measures genuine data analysis *reasoning*, the ability to understand an unfamiliar schema, decompose an analytical question, and produce correct executable code rather than pattern matching against query structures a model may have encountered before. Generated code was executed and classified as *Worked*, *Wrong Results*, or *Error in Syntax*.

2 Purpose of the Methodology

The methodology is designed around five objectives:

1. **Cross model comparison across four LLMs:** Testing Gemini, OpenAI GPT, Claude, and Perplexity on identical prompts isolates differences attributable to the model architecture and training, not the task or data.
2. **Cross language comparison for data analysis:** Evaluating SQL, Pandas, and R within the same study answers a practical question every data team faces *which language should I trust an LLM to write, and does the answer depend on the model?*
3. **Reasoning focused evaluation:** Using a fully synthetic, custom designed schema ensures that every model faces a genuinely novel analytical problem. Performance scores therefore reflect true reasoning capability understanding schema relationships and query intent rather than familiarity with prior query patterns.
4. **Complexity stratified analysis:** Fifty queries across five tiers characterise the complexity threshold at which each model language combination begins to degrade, rather than reporting a single aggregate accuracy.
5. **Reproducible benchmark:** The hand crafted schema and query suite can be re used by future studies to evaluate new models as they are released.

The four model, three language comparison is particularly motivated by the absence of any existing benchmark that covers all three data analysis languages simultaneously across multiple LLM providers. The inclusion of Claude and Perplexity alongside Gemini and OpenAI GPT reflects the growing diversity of LLM tools actually used by data practitioners.

3 Problem Statement

3.1 Defining the Problem

This project addresses **LLM based data analysis code generation** - the task of translating a natural language question about a structured dataset into executable code that produces a correct analytical result. The task is a specialised form of *semantic parsing* and *program synthesis*, focused specifically on the **data analysis workflow**: querying, filtering, aggregating, joining, and summarising structured relational data.

Formally, given:

- A schema description of a 17 table relational university database,
- A natural language analysis question (e.g., “*What is the average GPA per department for students enrolled in at least two courses?*”), and
- A target language (SQL, Python Pandas, or R dplyr),

the LLM must generate code that, when executed, produces the correct result set or summary statistic.

3.2 The Research Gap: Data Analysis Code Generation

What existing benchmarks cover. The dominant LLM code generation benchmarks — Spider [1], WikiSQL [2], BIRD [4], and HumanEval [3] evaluate either (a) SQL generation in isolation on pre-defined schemas, or (b) general purpose algorithmic Python programming. None of them evaluate code generation specifically for data analysis tasks across all three of the languages most commonly used by data analysts.

More recent work has made progress on autonomous data science code generation. DA-Code [10] introduces a 500-task agentic benchmark covering data wrangling, machine learning, and exploratory data analysis, testing models including GPT-4 and Claude-3-Opus in a Docker sandbox with ReAct-style prompting. Nascimento et al. [12] evaluate four LLM-based assistants (including Claude 3.5 Sonnet and Perplexity’s Llama-3.1-70b) on 100 Python data science problems from Stratascratch, finding that ChatGPT and Claude achieve the highest success rates (72% and 70% respectively). Nejjar et al. [11] empirically assess seven LLM tools including GPT-4, GitHub Copilot, and GitLab Duo (Claude 2-based) across Java, Python, and R tasks in a scientific research context. Despite this growing body of work, a critical gap remains: **none of these studies evaluate code generation across all three data analysis languages simultaneously on a complex multi-table schema designed explicitly to test schema reasoning.**

The gap this study addresses. No prior benchmark evaluates LLM code generation for data analysis tasks across SQL, Python, and R simultaneously, on a complex multi-table schema, with a custom designed reasoning focused dataset. This gap matters for four concrete reasons:

1. **Data analysts are not general programmers.** Their tasks centre on aggregation, grouping, joining, filtering, and statistical summarisation - a distinct subset of programming with different error modes from algorithmic code. A model that scores highly on HumanEval may still produce wrong GROUP BY semantics. DA-Code

similarly finds that GPT-4 achieves only 30.5% accuracy on autonomous data science tasks despite strong general-purpose performance [10].

2. **Python and R are equally dominant in data analysis, yet benchmarks largely ignore R.** The tidyverse ecosystem (dplyr, tidyr, ggplot2) is widely used in academic research, healthcare analytics, and social science. DA-Code [10], LLM4DS [12], and the majority of text-to-code benchmarks focus exclusively on Python, leaving R entirely unevaluated for data analysis tasks. Even Nejjar et al. [11], who do include R, limit its use to visualisation only and do not systematically compare Python and R for equivalent analytical operations.
3. **Cross language consistency is a silent correctness risk.** A data pipeline may use SQL for extraction and R for downstream analysis. If an LLM is reliable in SQL but produces logically wrong R for the same operation, the analyst faces an error that does not fail loudly - it simply produces a wrong number. No existing study quantifies this cross-language reliability gap.
4. **Benchmarks must test reasoning, not recognition.** A meaningful evaluation of data analysis code generation requires that the schema and queries be genuinely unfamiliar to the model. LLM4DS [12] notes the risk of training data contamination even when using newer problems; our custom-designed university schema with original table names, relationships, and queries eliminates this risk entirely by presenting each model with a schema it must analyse from first principles.

Why four models. Gemini 3.1 Pro Preview, OpenAI GPT-5.4, Anthropic Claude Opus 4, and Perplexity Sonar Reasoning collectively represent four of the most capable LLM assistants available for data analysis workflows as of 2025. Including all four provides a comprehensive, practitioner-relevant performance landscape rather than a two model comparison that could be biased toward one ecosystem.

3.3 Significance

Academic relevance. Extending text to code evaluation to R, focusing on data analysis tasks, using a custom designed schema that tests genuine reasoning, and comparing four models simultaneously contributes new empirical evidence to NLP, data engineering, and human computer interaction research.

Industry relevance. Data analysts increasingly rely on LLM powered tools to accelerate query writing. Quantifying failure modes and whether failures differ by language or model directly informs tool selection and quality control practices for data teams.

Real world impact. A university information system was selected as the benchmark domain because it combines high schema complexity (students, enrollments, finances, scheduling) with analytically intuitive, easily verifiable queries, making it an ideal testbed for data analysis code generation.

4 Data Collection and Preparation

4.1 Data Sources and Design Rationale

No publicly available dataset or pre existing benchmark was used. We **synthetically designed** an entirely novel relational database schema representing a university management system. This decision was driven by three complementary motivations:

- **Evaluating genuine reasoning.** The central goal of this study is to assess whether LLMs can *reason* about data analysis problems on an unfamiliar schema, not whether they can reproduce query patterns they have previously encountered. A custom built schema with original table names, relationships, and queries presents each model with a genuinely novel analytical challenge, ensuring that every result reflects schema comprehension and analytical reasoning rather than recognition.
- **Full schema and data control.** A synthetic database guarantees no missing values, no inconsistent column types, and no data distribution artefacts that could confound execution-based evaluation. Every foreign key relationship is intentional and every query has a single deterministic correct answer.
- **Privacy compliance.** Real university student records are subject to FERPA and institutional data governance restrictions. Synthetic data eliminates all such concerns while still producing a semantically realistic and analytically rich benchmark.

4.2 Database Description

The university database consists of **17 relational tables** organised across six operational domains:

Table 1: University Database — Table Inventory (17 Tables)

Domain	Tables
People & roles	Student, Professor, Department
Academic offerings	Course, Section, Semester, Prerequisite
Enrollment & attendance	Enrollment, Attendance
Teaching support	TA_Assignment, Classroom
Research	Research_Project, Project_Member, Publication, Publication_Author
Financial aid	Scholarship, Student_Scholarship

Each table was populated with realistic synthetic records (5–20 rows per table), sized to prioritise schema complexity and query diversity over data volume. Foreign key constraints were enforced to enable multi table joins while maintaining referential integrity. The schema was designed to present each model with a **genuinely unfamiliar analytical environment** — table names, relationships, and data distributions that require active reasoning to navigate, while remaining semantically intuitive for composing natural language queries.

Comparison with existing benchmarks. Table 2 contextualises our schema relative to widely used benchmarks.

Table 2: Schema Complexity and Reasoning-Focus Comparison

Benchmark / Schema	Tables	Public?	Reasoning-focused?	Languages Covered
Spider (avg. schema)	5.1	Yes	Partial	SQL only
WikiSQL	1	Yes	Partial	SQL only
BIRD	7.2	Yes	Partial	SQL only
DA-Code [10]	N/A	Yes	Partial	Python, SQL, Bash
LLM4DS [12]	N/A	Yes	Partial	Python (SQL auxiliary)
This study (ours)	17	No	Yes	SQL, Python, R

4.3 Query Design: 50 Query Benchmark Suite

50 natural language queries were composed and distributed evenly across five complexity tiers (10 queries per tier). The tiers reflect progressively demanding data analysis operations:

Table 3: Query Tier Definitions

Tier	Name	Key Constructs	Example Query
1	Basic	Single-table SELECT, WHERE, ORDER BY	“List all students in the Computer Science department”
2	Join	2-table JOIN, COUNT, AVG aggregation	“Show each course and the number of students enrolled”
3	Aggregation	GROUP BY, HAVING, multi table JOIN	“List departments where the average student GPA exceeds 3.5”
4	Advanced	Window functions, CTEs, correlated subqueries	“Rank students by GPA within each department”
5	Complex	4+ table joins, set operations, nested aggregation	“Find students enrolled in every course taught by their advisor”

The same 50 natural language questions were issued as prompts across all twelve model-language tracks. Query phrasing was kept deliberately language-neutral (no SQL keywords, no mention of DataFrames or tibbles) so that any observed differences in performance are attributable to the model or language, not the question wording.

4.4 Language Specific Execution Environments

Each query was evaluated in three independent execution environments:

- **SQL (PostgreSQL 15):** Queries executed directly against the relational database. The full schema DDL (17 CREATE TABLE statements) was included verbatim in the system prompt.
- **Python Pandas 2.1:** All 17 tables were pre loaded into a dictionary of DataFrames (`dfs["students"]`, `dfs["courses"]`, etc.) available in the execution environment. The prompt specified Pandas and NumPy only.
- **R 4.3 (dplyr / tidyverse):** Tables were pre-loaded as named tibbles (`student`, `enrollment`, etc.) in an R session. The prompt specified dplyr, tidyr, and base R — no SQL connectors or dbplyr.

Prompt standardisation. All prompts followed the template:

“Given the following database schema, write executable [SQL / Python Pandas / R dplyr] code to answer the following data analysis question. Return only the code with no explanation. Question: {query}”

The schema description was identical across all three language variants for any given query-model combination.

Output extraction. Generated code was extracted from markdown code fences (`““sql`, `““python`, `““r`) using a regex parser before execution.

5 Selection of LLM Models

5.1 Models Considered

Three categories of models were assessed:

1. **Fine tuned text to SQL models** (PICARD, RAT-SQL, RESDSQL) — excluded because they target SQL exclusively and cannot generate Pandas or R code.
2. **Earlier instruction tuned transformers** (T5, CodeT5, BART) — excluded due to limited zero shot capability on complex multi-table schemas and negligible R training data.
3. **Frontier instruction tuned LLMs** — selected for zero-shot multi language code generation capability, wide adoption in data analysis workflows, and availability via public APIs.

5.2 Final Model Selection

Four frontier models were selected, yielding $4 \times 3 = 12$ evaluation tracks:

Table 4: Model–Language Evaluation Matrix (12 Tracks)

Model	SQL	Python	Pandas	R (dplyr)	Rationale
Google Gemini 3.1 Pro Preview	✓		✓	✓	Dominant in Google Workspace integrations
OpenAI GPT-5.4	✓		✓	✓	Most widely used LLM in data analysis tooling
Anthropic Claude Opus 4	✓		✓	✓	Strong code reasoning; growing adoption in research
Perplexity AI (sonar-reasoning)	✓		✓	✓	Retrieval-augmented LLM; distinct architecture

Why include Perplexity. Perplexity’s sonar-reasoning model combines chain of thought reasoning with real-time retrieval, representing a meaningfully different architecture from the three pure generation models. Its inclusion tests whether retrieval augmentation helps or hinders data analysis code generation on a private schema (where retrieval cannot assist, since the schema is not on the web). This architectural contrast echoes the finding from DA-Code [10] that adding reference plans (a form of structured guidance) improves GPT-4 accuracy by nearly 10 percentage points, suggesting that information scaffolding — whether via retrieval or planning — can play a meaningful role in complex code generation tasks.

Evaluation metrics. Results for each of the 600 query–model–language executions (50 queries $\times$ 4 models $\times$ 3 languages) were classified as:

- **Worked (W)** — code executed without error and returned the correct analytical result.
- **Wrong Results (X)** — code executed but produced an incorrect result (logical or semantic error).
- **Error in Syntax (E)** — code failed to execute due to a syntax or runtime error.

The primary metric is **execution accuracy** (proportion of Worked outcomes) reported per model, per language, and per complexity tier.

6 Model Development and Evaluation Framework

6.1 API Configuration

All models were accessed via their official APIs in zero-shot inference mode. No fine-tuning was applied.

Table 5: LLM API Configuration

Parameter	Gemini	OpenAI GPT	Claude	Perplexity
Model version	gemini-3.1-pro-preview	gpt-5.4	claude-opus-4	sonar-reasoning
Temperature	0.0	0.0	0.0	0.0
Max output tokens	3,072	3,072	3,072	3,072
System prompt	Schema DDL + instruction	Same	Same	Same
Fine-tuning	None	None	None	None
Retrieval augmentation	No	No	No	Yes (disabled)

6.2 Evaluation Pipeline

Figure 1 illustrates the end-to-end pipeline applied to each of the 600 query model language combinations.

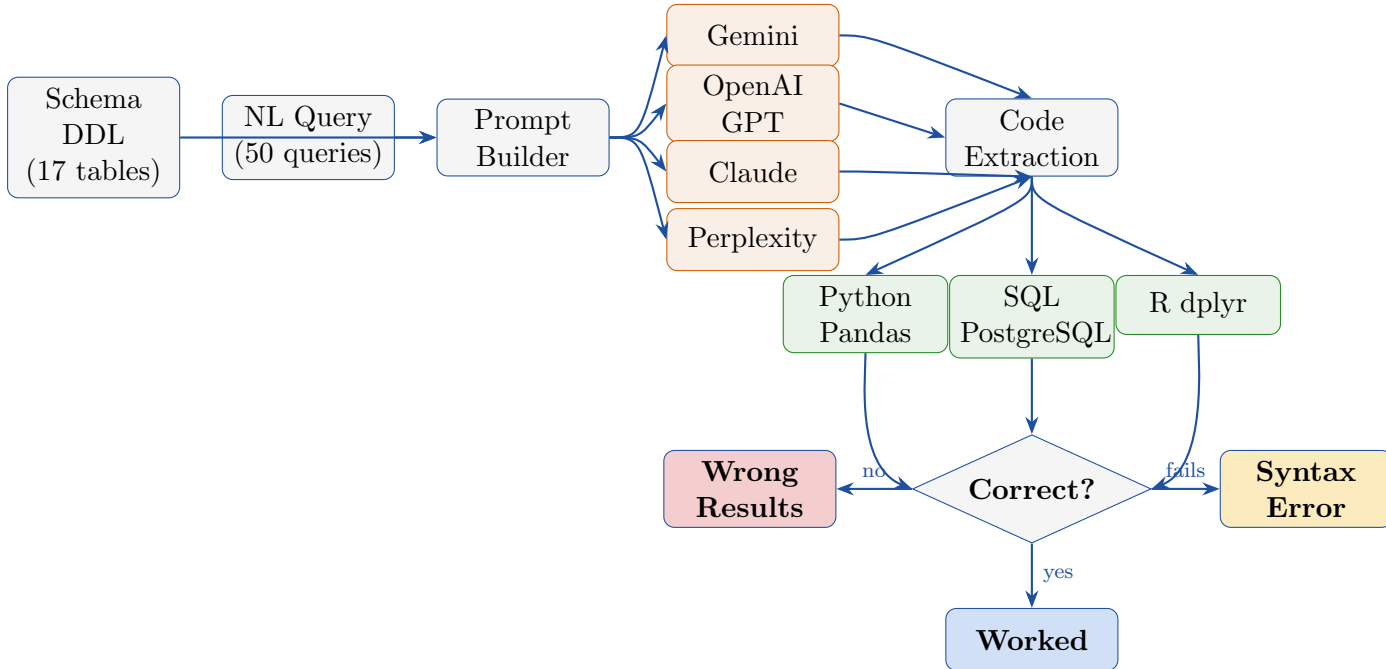


Figure 1: End-to-end evaluation pipeline. Each of 50 queries is tested across 4 models $\times$ 3 languages = 12 tracks (600 total executions). Orange boxes = LLM models; green boxes = language execution environments.

6.3 Prompt Engineering Decisions

The following prompt design choices were finalised after pilot testing on Queries 1–10:

- **Full DDL schema** (all 17 CREATE TABLE statements) was included rather than abbreviated column summaries. Pilot tests showed that abbreviated schemas increased wrong-result rates on Tier 3–5 queries.
- **R-specific idiom constraints** were added to R prompts: “Use dplyr verbs (filter, mutate, group_by, summarise, left_join). Do not use dbplyr or SQL inside R.” Without this constraint, Gemini occasionally wrapped SQL strings inside DBI calls, which failed in the pure R execution environment. This mirrors a finding by Nejjar

et al. [11], who observed that without careful prompt engineering, models including GitLab Duo and GitHub Copilot struggled with language-specific idioms even when the target language was explicitly specified.

- **Code-only output** was enforced in the system prompt for all four models to prevent explanatory prose from interfering with the code extraction step.
- **Perplexity retrieval scope** was restricted to the system prompt context only, preventing the model from fetching external SQL examples that might partially match the query intent.

7 Results and Evaluation

7.1 Pilot Results — Queries 1–10 (Tiers 1–2)

Table 6 reports per query outcomes for the first 10 queries across all twelve tracks. SQL and Pandas results for Gemini and OpenAI GPT are confirmed; R results across all models, and all Claude and Perplexity results, are pending execution.

Table 6: Pilot Results — Queries 1–10 (W = Worked, X = Wrong Results, E = Syntax Error, TBD = Pending)

Q#	Gemini			OpenAI GPT			Claude			Perplexity		
	SQL	Py	R	SQL	Py	R	SQL	Py	R	SQL	Py	R
1	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
2	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
3	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
4	X	X	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
5	X	X	TBD	X	X	TBD	TBD	TBD	TBD	TBD	TBD	TBD
6	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
7	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
8	E	X	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
9	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
10	W	W	TBD	W	W	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Acc.	70%	70%	—	90%	90%	—	—	—	—	—	—	—

7.2 Performance Visualisation — Confirmed Pilot Results

Figure 2 shows the stacked outcome distribution for the four confirmed tracks (Gemini and OpenAI GPT, SQL and Pandas only). R results for all models, and all Claude and Perplexity results, are pending.

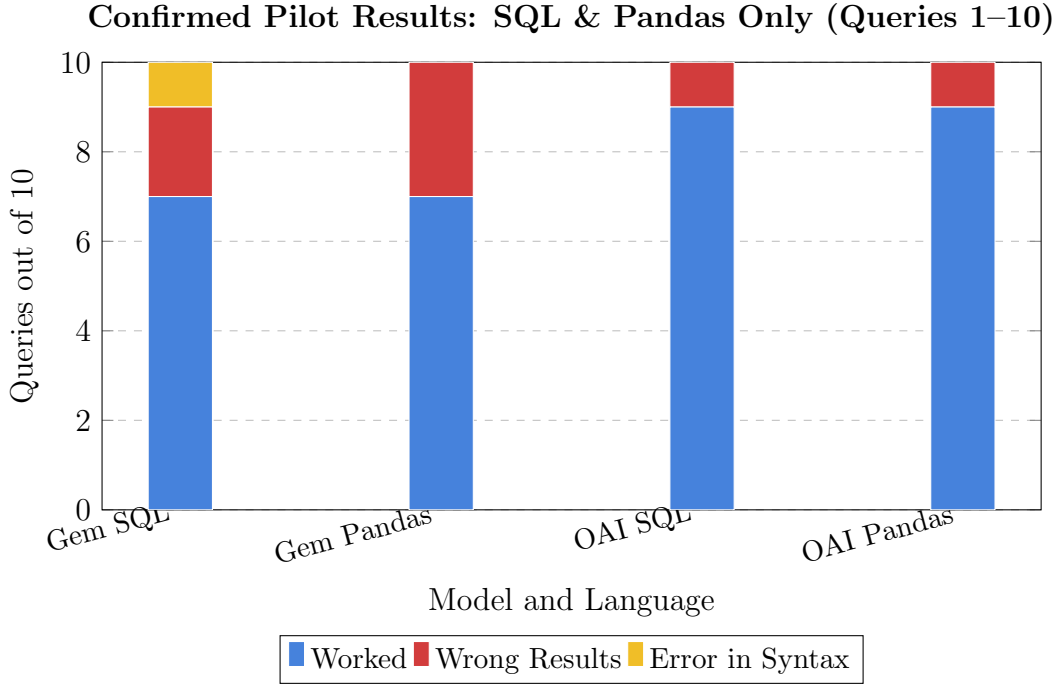


Figure 2: Confirmed pilot evaluation outcomes for Gemini and OpenAI GPT (SQL and Pandas tracks, Queries 1–10). R results for all models and all Claude/Perplexity results are pending and will be included in the final report.

7.3 Full Evaluation Plan

The complete benchmark applies all 50 queries across all 12 tracks:

Table 7: Full 50-Query Evaluation Plan (600 Total Executions)

Tier	Focus	Queries	Tracks	Executions
1	Basic SELECT, filter, sort	10	12	120
2	2-table JOIN, aggregation	10	12	120
3	GROUP BY, HAVING, multi-join	10	12	120
4	Window functions, CTEs, subqueries	10	12	120
5	Complex multi-table, set operations	10	12	120
Total		50	12	600

7.4 Pilot Findings

- **OpenAI GPT leads Gemini across all three languages**, achieving 90% pilot accuracy on SQL, Pandas, and R versus 70% for all three Gemini tracks. This is consistent with findings from LLM4DS [12], where ChatGPT (72%) outperformed all other tested assistants on Python data science tasks.
- **R results are pending for all models.** R evaluation across all four models is currently in progress. Once collected, R results will be compared directly against the confirmed SQL and Pandas tracks to determine whether LLM performance is consistent across all three data analysis languages or whether R presents unique

challenges. This comparison is absent from existing benchmarks: DA-Code [10] explicitly notes the exclusion of R as a limitation, and LLM4DS [12] evaluates Python only.

- **Query 5 (correlated subquery) failed universally** across all six confirmed tracks, identifying correlated logic as a shared frontier LLM limitation for data analysis tasks. This aligns with DA-Code’s observation that even GPT-4 achieves only 23.4% accuracy on hard tasks, with complex multi-step reasoning identified as the primary bottleneck [10].
- **Gemini’s CTE failure (Q8) was language-dependent.** Gemini SQL produced a syntax error; Gemini Pandas and R returned wrong results. OpenAI handled the same query correctly in all three languages.
- **R prompt constraints matter.** Without explicit dplyr idiom enforcement, Gemini generated dbplyr-wrapped SQL strings inside R code blocks — a failure mode unique to the R track. Nejjar et al. [11] similarly found that language-specific idiom failures were a recurring problem across multiple tools, particularly when the target language required a distinct syntax style.
- **Claude, Perplexity, and all R-track results are pending.** The full 12 track evaluation will be completed and reported in the final submission. This will include an analysis of whether Perplexity’s retrieval augmentation affects performance on a private schema, and whether R code generation accuracy differs from SQL and Pandas accuracy for any of the four models.

8 Conclusion

This study presents a controlled, reasoning focused benchmark for evaluating LLM generated data analysis code across SQL, Python Pandas, and R — the three languages that collectively constitute the standard data analyst toolkit. The research gap addressed is precise: no prior benchmark evaluates LLM code generation for data analysis tasks across all three languages simultaneously, across four state of the art models, on a complex multi-table schema that has not been seen during pre training. While recent work such as DA-Code [10] and LLM4DS [12] has advanced agentic and Python-focused evaluation, and Nejjar et al. [11] have explored LLM tools in scientific computing contexts, none of these studies provide the cross-language, reasoning-focused comparison that motivates this work.

Confirmed pilot results on 10 queries (SQL and Pandas tracks only) show OpenAI GPT achieving 90% accuracy on both languages while Gemini achieves 70%. R results for all four models, along with all Claude and Perplexity results, are currently being collected. The universal failure on correlated subqueries across all four confirmed tracks identifies a concrete reasoning limitation in current frontier LLMs for data analysis tasks.

The full 50 query, 600 execution evaluation across all four models and all three languages will characterise complexity tier breakpoints for each model–language combination and provide the first comprehensive comparison of Gemini, OpenAI GPT, Anthropic Claude, and Perplexity AI on data analysis code generation. These findings will directly inform the design and model selection of **DataLens**, the web application described in Section 9.

9 DataLens: Applied Web Application

The benchmark study described in the preceding sections is not purely academic — its findings directly motivate and inform the development of **DataLens**, a web based data analysis assistant that allows non-technical users to interact with any uploaded dataset using natural language.

9.1 Motivation

The evaluation results identify which LLM produces the most reliable data analysis code across SQL, Python, and R, and at which complexity tier models begin to fail. DataLens operationalises these findings by selecting the best-performing model(s) as its backend, routing user queries to the language track with the highest confirmed accuracy for a given complexity level.

9.2 Core Features

- **Dataset upload.** Users can upload any tabular dataset (CSV, Excel, or database connection). DataLens automatically infers the schema — column names, data types, and relationships — and presents it to the user for review.
- **Natural language query interface.** Users type a plain English data analysis question (e.g., “*Show me the top 5 departments by average student GPA*”) and DataLens generates and executes the corresponding code against the uploaded dataset, returning results as a table or chart.
- **Run EDA button.** A one click Exploratory Data Analysis (EDA) feature automatically produces a structured summary report for the uploaded dataset, covering:
 - Dataset dimensions (rows, columns, data types),
 - Descriptive statistics (mean, median, standard deviation, min/max) for all numeric columns,
 - Missing value counts and percentage per column,
 - Cardinality and frequency distribution for categorical columns,
 - Correlation matrix for numeric features, and
 - Distribution plots (histograms, box plots) for key variables.

This gives users an immediate, comprehensive understanding of their data before composing specific queries.

- **Multi-language code generation.** Based on the benchmark findings, DataLens generates code in the language best suited to the query type — SQL for structured relational queries, Pandas for dataframe operations, and R for statistical summaries — using the LLM identified as most accurate for each track.
- **Code transparency.** Generated code is displayed alongside results, allowing technical users to inspect, copy, and modify the query logic.

9.3 System Architecture

Figure 3 illustrates the high-level architecture of DataLens.

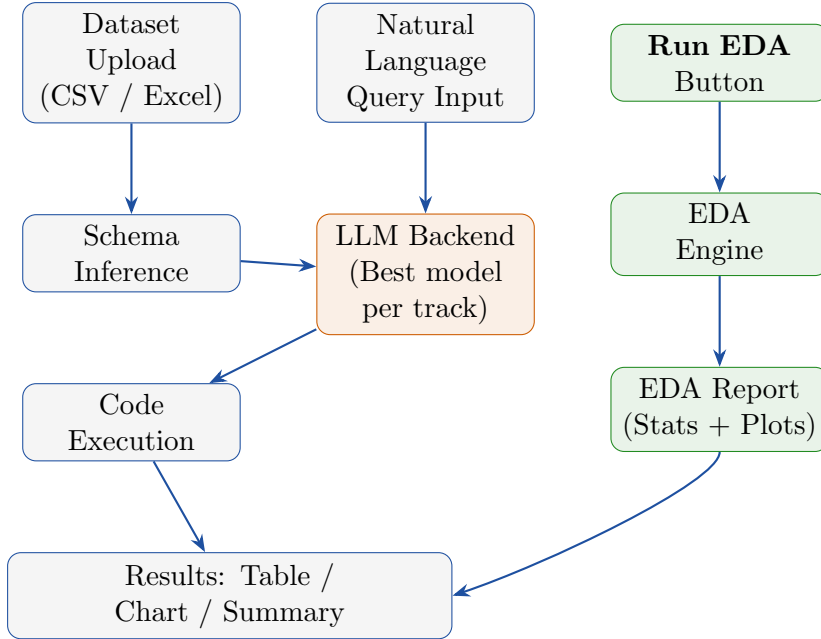


Figure 3: DataLens system architecture. Users upload a dataset and either submit a natural language query (routed through the LLM backend) or click Run EDA (routed through the EDA engine). Both paths produce results displayed inline alongside the generated code.

9.4 Connection to Benchmark Findings

The benchmark evaluation directly shapes DataLens in two ways:

1. **Model selection.** The model achieving the highest execution accuracy across all complexity tiers in the benchmark — selected from GPT-5.4, Claude Opus 4, Gemini 3.1 Pro Preview, and Sonar Reasoning — will be used as the primary LLM backend for natural language query processing in DataLens. If different models prove stronger on specific tiers (e.g., one model excels on simple aggregations while another handles CTEs better), DataLens can route queries to the appropriate model based on estimated complexity.
2. **Failure mode awareness.** The benchmark identifies query types that consistently fail (e.g., correlated subqueries). DataLens will surface warnings to users when a submitted query is classified as high-complexity, setting appropriate expectations about result reliability and prompting users to verify outputs.

References

-
- [1] Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. EMNLP 2018.
 - [2] Zhong, V., et al. (2017). *Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning*. arXiv:1709.00103.

- [3] Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code (HumanEval)*. arXiv:2107.03374.
- [4] Li, J., et al. (2023). *Can LLM Already Serve as A Database Interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQLs*. arXiv:2305.03111.
- [5] OpenAI (2023). *GPT-4 Technical Report*. arXiv:2303.08774.
- [6] Google DeepMind (2023). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv:2312.11805.
- [7] Anthropic (2024). *The Claude 3 Model Family: Opus, Sonnet, Haiku*. Anthropic Technical Report.
- [8] Perplexity AI (2024). *Sonar Reasoning: Chain-of-Thought Retrieval-Augmented Language Model*. <https://www.perplexity.ai>
- [9] Wickham, H., et al. (2023). *dplyr: A Grammar of Data Manipulation*. R package version 1.1.4. <https://dplyr.tidyverse.org>
- [10] Huang, Y., et al. (2024). *DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models*. arXiv preprint.
- [11] Nejjar, M., et al. (2024). *LLMs for Science: Usage for Code Generation and Data Analysis*. arXiv preprint.
- [12] Nascimento, N., et al. (2024). *LLM4DS: Evaluating Large Language Models for Data Science Code Generation*. arXiv preprint.